

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	VIMALA K	Reg. No.:	192424276
Section / Batch:	B.tech AI&DS	Date:	15/8/26

Code of Conduct

I **VIMALA K 192424276** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Coding:

```
import nltk

nltk.download("punkt")

words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]

analyses = {
    "treatment": ["", "treat", "ment"],
    "treatable": ["", "treat", "able"],
    "retreatment": ["re", "treat", "ment"],
    "treated": ["", "treat", "ed"],
    "untreated": ["un", "treat", "ed"]
}

affix_type = {
    "re": "Derivational",
    "un": "Derivational",
    "ment": "Derivational",
    "able": "Derivational",
    "ed": "Inflectional"
}

print("=" * 70)
print("QUESTION 1 - BIOMEDICAL MORPHOLOGICAL ANALYSIS")
print("=" * 70)

for word in words:

    prefix = analyses[word][0]
    root = analyses[word][1]
    suffix = analyses[word][2]

    print("\nWord:", word)
```

```

if prefix:
    print("Prefix:", prefix)
    print("Prefix Type:", affix_type[prefix])
else:
    print("Prefix: None")

print("Root:", root)

if suffix:
    print("Suffix:", suffix)
    print("Suffix Type:", affix_type[suffix])
else:
    print("Suffix: None")

parts = []

if prefix:
    parts.append(prefix)

parts.append(root)

if suffix:
    parts.append(suffix)

print("Decomposition:", " + ".join(parts))

print("\nSUMMARY")

print("treatment = treat + ment")
print("treatable = treat + able")
print("retreatment = re + treat + ment")
print("treated = treat + ed")
print("untreated = un + treat + ed")

print("\nInterpretation")
print("Derivational affixes change meaning or word class.")
print("Inflectional affixes express grammatical information.")
print("Correct morphological analysis improves biomedical search.")

```

The screenshot shows a Python IDE with two windows. The left window contains the following code:

```
import nltk

nltk.download("punkt")

words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]

analyses = {
    "treatment": ["", "treat", "ment"],
    "treatable": ["", "treat", "able"],
    "retreatment": ["re", "treat", "ment"],
    "treated": ["", "treat", "ed"],
    "untreated": ["un", "treat", "ed"]
}

affix_type = {
    "re": "Derivational",
    "un": "Derivational",
    "ment": "Derivational",
    "able": "Derivational",
    "ed": "Inflectional"
}

print("=" * 70)
print("QUESTION 1 - BIOMEDICAL MORPHOLOGICAL ANALYSIS")
print("=" * 70)

for word in words:
    prefix = analyses[word][0]
    root = analyses[word][1]
```

The right window shows the output of the code:

```
SUFFIX: able
Suffix Type: Derivational
Decomposition: treat + able

Word: retreatment
Prefix: re
Prefix Type: Derivational
Root: treat
Suffix: ment
Suffix Type: Derivational
Decomposition: re + treat + ment

Word: treated
Prefix: None
Root: treat
Suffix: ed
Suffix Type: Inflectional
Decomposition: treat + ed

Word: untreated
Prefix: un
Prefix Type: Derivational
Root: treat
Suffix: ed
Suffix Type: Inflectional
Decomposition: un + treat + ed

SUMMARY
treatment = treat + ment
treatable = treat + able
retreatment = re + treat + ment
treated = treat + ed
untreated = un + treat + ed
```

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Coding:

```
prefixes = [
    "un",
    "re",
    "dis"
]
```

```
suffixes = [
    "able",
    "ing",
    "ed",
]
```

```

    "er",
    "s"
]

words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]

roots = {
    "replayed": "play",
    "unhappier": "happy",
    "disconnected": "connect",
    "players": "play",
    "restarting": "start",
    "unreadable": "read"
}

def parser(word):

    original = word
    found_prefixes = []
    found_suffixes = []

    changed = True

    while changed:

        changed = False

        for prefix in prefixes:

            if word.startswith(prefix):

                if len(word) > len(prefix) + 2:

                    found_prefixes.append(prefix)

                    word = word[len(prefix):]

                    changed = True

                    break

        changed = True

    while changed:

        changed = False

        for suffix in sorted(
            suffixes,
            key=len,
            reverse=True
        ):

            if word.endswith(suffix):

                if len(word) > len(suffix) + 2:

```

```

        found_suffixes.insert(
            0,
            suffix
        )

        word = word[:-len(suffix)]

        changed = True

        break

    return (
        original,
        found_prefixes,
        roots.get(original, word),
        found_suffixes
    )

```

```

print("=" * 70)
print("QUESTION 2 - FINITE-STATE MORPHOLOGICAL PARSER")
print("=" * 70)

```

for word in words:

```

    original, prefixes_found, root, suffixes_found = parser(word)

```

```

    parts = []

```

```

    for item in prefixes_found:
        parts.append(item)

```

```

    parts.append(root)

```

```

    for item in suffixes_found:
        parts.append(item)

```

```

    print("\nWord:", original)
    print("Prefixes:", prefixes_found)
    print("Root:", root)
    print("Suffixes:", suffixes_found)
    print("Analysis:", " " + ".join(parts))

```

```

print("\nPARSER STRUCTURE")
print("START -> PREFIX* -> ROOT -> SUFFIX* -> END")

```

```

print("\nAccuracy Test")

```

```

correct = 0

```

```

expected = {
    "replayed": ["re", "play", "ed"],
    "unhappier": ["un", "happy", "er"],
    "disconnected": ["dis", "connect", "ed"],
    "players": ["play", "er", "s"],
    "restarting": ["re", "start", "ing"],
    "unreadable": ["un", "read", "able"]
}

```

for word in words:

```

    original, p, root, s = parser(word)

```

```

    result = p + [root] + s

```

```

if result == expected[word]:
    correct += 1

```

```

accuracy = correct / len(words) * 100

```

```

print("Correct Analyses:", correct)
print("Total Words:", len(words))
print("Accuracy:", round(accuracy, 2), "%")

```

co2 at3 q2.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at3 q2.py (3.14.3)

```

def parse_word(word):
    parts = []
    prefixes_found = []
    suffixes_found = []
    root = word
    while root and root[-1] in 'aeiouy':
        prefixes_found.append(root[-1])
        root = root[:-1]
    while root and root[0] in 'aeiouy':
        suffixes_found.append(root[0])
        root = root[1:]
    parts.append(item)

    print("\nWord:", original)
    print("Prefixes:", prefixes_found)
    print("Root:", root)
    print("Suffixes:", suffixes_found)
    print("Analysis:", " + ".join(parts))

print("\nPARSER STRUCTURE")
print("START -> PREFIX* -> ROOT -> SUFFIX* -> END")

print("\nAccuracy Test")

correct = 0

expected = {
    "replayed": ["re", "play", "ed"],
    "unhappier": ["un", "happy", "er"],
    "disconnected": ["dis", "connect", "ed"],
    "players": ["play", "er", "s"],
    "restarting": ["re", "start", "ing"],
    "unreadable": ["un", "read", "able"]
}

for word in words:
    original, p, root, s = parser(word)

    result = p + [root] + s

    if result == expected[word]:
        correct += 1

accuracy = correct / len(words) * 100

print("Correct Analyses:", correct)

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Word: unhappier
Prefixes: ['un']
Root: happy
Suffixes: ['er']
Analysis: un + happy + er

Word: disconnected
Prefixes: ['dis']
Root: connect
Suffixes: ['ed']
Analysis: dis + connect + ed

Word: players
Prefixes: []
Root: play
Suffixes: ['er', 's']
Analysis: play + er + s

Word: restarting
Prefixes: ['re']
Root: start
Suffixes: ['ing']
Analysis: re + start + ing

Word: unreadable
Prefixes: ['un', 're']
Root: read
Suffixes: []
Analysis: un + re + read

PARSER STRUCTURE
START -> PREFIX* -> ROOT -> SUFFIX* -> END

```

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```

from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())

```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

Coding:

```
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

documents = [
    "The market is growing rapidly.",
    "The company announced a new product.",
    "Technology companies are expanding.",
    "The investors are studying the financial report.",
    "The organization organized a new meeting."
]

def process_text(text):
    tokens = re.findall(r"[a-zA-Z]+", text.lower())

    stemmed = []

    for token in tokens:
        stemmed.append(ps.stem(token))

    return tokens, stemmed

print("=" * 60)
print("QUESTION 3 - STEMMING ERROR ANALYSIS")
print("=" * 60)

for i, text in enumerate(documents):

    tokens, stemmed = process_text(text)

    print("\nDOCUMENT", i + 1)

    print("Original Text:")
    print(text)

    print("\nTokens:")
    print(tokens)

    print("\nStemmed Tokens:")
    print(stemmed)

print("\nPROBLEMATIC STEMMING CASES")

examples = [
    "connected",
    "connection",
    "connecting",
    "connectivity",
    "studies",
    "studied",
    "studying",
    "study",
    "organize",
    "organized",
    "organizer",
    "organization",
    "treatment",
    "treated",
```

```

    "readable",
    "happiness",
    "relational",
    "usefulness",
    "agreement",
    "running"
]

for word in examples:
    print(word, "->", ps.stem(word))

print("\n" + "=" * 60)
print("PROCESSING COMPLETED")
print("=" * 60)

```

co2_at3 q3.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2_at3 q3.py (3.14.3)

File Edit Format Run Options Window Help

```

print(tokens)

print("\nStemmed Tokens:")
print(stemmed)

print("\nPROBLEMATIC STEMMING CASES")

examples = [
    "connected",
    "connection",
    "connecting",
    "connectivity",
    "studies",
    "studied",
    "studying",
    "study",
    "organize",
    "organized",
    "organizer",
    "organization",
    "treatment",
    "treated",
    "readable",
    "happiness",
    "relational",
    "usefulness",
    "agreement",
    "running"
]

for word in examples:
    print(word, "->", ps.stem(word))

print("\n" + "=" * 60)
print("PROCESSING COMPLETED")
print("=" * 60)

```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
['the', 'investor', 'are', 'studi', 'the', 'financi', 'report']
```

DOCUMENT 5

Original Text:

The organization organized a new meeting.

Tokens:

```
['the', 'organization', 'organized', 'a', 'new', 'meeting']
```

Stemmed Tokens:

```
['the', 'organ', 'organ', 'a', 'new', 'meet']
```

PROBLEMATIC STEMMING CASES

connected -> connect

connection -> connect

connecting -> connect

connectivity -> connect

studies -> studi

studied -> studi

studying -> studi

study -> studi

organize -> organ

organized -> organ

organizer -> organ

organization -> organ

treatment -> treatment

treated -> treat

readable -> readabl

happiness -> happi

relational -> relat

usefulness -> use

agreement -> agreement

running -> run

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```

def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))

```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Coding:

```
import nltk
```

```
nltk.download("wordnet")
nltk.download("omw-1.4")
```

```
from nltk.corpus import wordnet
```

```
irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "geese": "goose",
    "feet": "foot",
    "teeth": "tooth",
    "people": "person"
}
```

```
def old_parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

```
def parser(word):

    if word in irregular:
        return irregular[word], "Plural Noun"

    if word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"

    if word.endswith("es"):
        return word[:-2], "Plural Noun"

    if word.endswith("s"):
        return word[:-1], "Plural Noun"

    return word, "Singular"
```

```
words = [
    "cars",
```

```

    "boxes",
    "cities",
    "children",
    "books",
    "men",
    "women",
    "mice",
    "dogs",
    "classes"
]

print("=" * 60)
print("QUESTION 4 - PLURAL NOUN PARSER")
print("=" * 60)

print("\nORIGINAL PARSER OUTPUT")

for word in words:
    root, tag = old_parser(word)
    print(word, "->", root, ",", tag)

print("\nCORRECTED PARSER OUTPUT")

for word in words:
    root, tag = parser(word)
    print(word, "->", root, ",", tag)

print("\nWORDNET CHECK")

for word in words:

    synsets = wordnet.synsets(
        word,
        pos=wordnet.NOUN
    )

    if len(synsets) > 0:
        print(word, "-> WordNet entry found")
    else:
        print(word, "-> WordNet entry not found")

print("\nRULES")

print("Regular plural: books -> book")
print("ES plural: boxes -> box")
print("IES plural: cities -> city")
print("Irregular plural: children -> child")

print("\n" + "=" * 60)

```

```

co2 at3 q4.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at3
File Edit Format Run Options Window Help

print("=" * 60)

print("\nORIGINAL PARSER OUTPUT")

for word in words:
    root, tag = old_parser(word)
    print(word, "->", root, ",", tag)

print("\nCORRECTED PARSER OUTPUT")

for word in words:
    root, tag = parser(word)
    print(word, "->", root, ",", tag)

print("\nWORDNET CHECK")

for word in words:
    synsets = wordnet.synsets(
        word,
        pos=wordnet.NOUN
    )

    if len(synsets) > 0:
        print(word, "-> WordNet entry found")
    else:
        print(word, "-> WordNet entry not found")

print("\nRULES")

print("Regular plural: books -> book")
print("ES plural: boxes -> box")
print("IES plural: cities -> city")
print("Irregular plural: children -> child")

print("\n" + "=" * 60)

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

books -> book , Plural Noun
men -> men , Singular
women -> women , Singular
mice -> mice , Singular
dogs -> do , Plural Noun
classes -> class , Plural Noun

CORRECTED PARSER OUTPUT
cars -> car , Plural Noun
boxes -> box , Plural Noun
cities -> city , Plural Noun
children -> child , Plural Noun
books -> book , Plural Noun
men -> man , Plural Noun
women -> woman , Plural Noun
mice -> mouse , Plural Noun
dogs -> dog , Plural Noun
classes -> class , Plural Noun

WORDNET CHECK
cars -> WordNet entry found
boxes -> WordNet entry found
cities -> WordNet entry found
children -> WordNet entry found
books -> WordNet entry found
men -> WordNet entry found
women -> WordNet entry found
mice -> WordNet entry found
dogs -> WordNet entry found
classes -> WordNet entry found

RULES
Regular plural: books -> book
ES plural: boxes -> box
IES plural: cities -> city
Irregular plural: children -> child

```

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

```

```

stemmer = PorterStemmer()

```

```

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

```

```

vectorizer = CountVectorizer()

```

```

X = vectorizer.fit_transform(documents)

```

```

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]

```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:

- Vocabulary size before correction
- Vocabulary size after correction
- Classification accuracy before correction
- Classification accuracy after correction
- Processing time

6. Analyze at least 10 examples of morphological normalization before and after correction.

7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Coding:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
import re
import time

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studied studying study",
    "organize organized organizer organization"
]

def normalize(text):
    words = re.findall(
        r"[a-zA-Z]+",
        text.lower()
    )

    result = []

    for word in words:
        result.append(stemmer.stem(word))

    return " ".join(result)

print("=" * 60)
print("QUESTION 5 - MORPHOLOGICAL FEATURE EXTRACTION")
print("=" * 60)

print("\nORIGINAL DOCUMENTS")

for document in documents:
    print(document)

start_before = time.time()

vectorizer_before = CountVectorizer()

X_before = vectorizer_before.fit_transform(
    documents
)

time_before = time.time() - start_before

vocabulary_before = (
    vectorizer_before.get_feature_names_out()
)
```

```

print("\nVOCABULARY BEFORE NORMALIZATION")

print(list(vocabulary_before))

print(
    "Vocabulary Size:",
    len(vocabulary_before)
)

start_after = time.time()

normalized_documents = []

for document in documents:
    normalized_documents.append(
        normalize(document)
    )

vectorizer_after = CountVectorizer()

X_after = vectorizer_after.fit_transform(
    normalized_documents
)

time_after = time.time() - start_after

vocabulary_after = (
    vectorizer_after.get_feature_names_out()
)

print("\nNORMALIZED DOCUMENTS")

for document in normalized_documents:
    print(document)

print("\nVOCABULARY AFTER NORMALIZATION")

print(list(vocabulary_after))

print(
    "Vocabulary Size:",
    len(vocabulary_after)
)

print("\nPROCESSING TIME")

print(
    "Before Normalization:",
    round(time_before, 6),
    "seconds"
)

print(
    "After Normalization:",
    round(time_after, 6),
    "seconds"
)

print("\nMORPHOLOGICAL NORMALIZATION")

examples = [
    "connected",
    "connection",
    "connecting",

```

```

    "connectivity",
    "studies",
    "studied",
    "studying",
    "study",
    "organize",
    "organized",
    "organizer",
    "organization"
]

for word in examples:
    print(
        word,
        "->",
        stemmer.stem(word)
    )

print("\nCOMPARISON")

print(
    "Vocabulary before:",
    len(vocabulary_before)
)

print(
    "Vocabulary after:",
    len(vocabulary_after)
)

reduction = (
    len(vocabulary_before)
    - len(vocabulary_after)
)

print(
    "Vocabulary reduction:",
    reduction
)

print("\nCONCLUSION")

print(
    "Morphological normalization is performed before"
)

print(
    "feature extraction."
)

print(
    "Related morphological forms can share a common stem."
)

print(
    "This reduces redundant vocabulary and can improve"
)

print(
    "machine-learning feature representation."
)

print("\n" + "=" * 60)

```

co2 at3 q5.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co2 at3
File Edit Format Run Options Window Help

```
len(vocabulary_after)
)

reduction = (
    len(vocabulary_before)
    - len(vocabulary_after)
)

print(
    "Vocabulary reduction:",
    reduction
)

print("\nCONCLUSION")

print(
    "Morphological normalization is performed before"
)

print(
    "feature extraction."
)

print(
    "Related morphological forms can share a common stem"
)

print(
    "This reduces redundant vocabulary and can improve"
)

print(
    "machine-learning feature representation."
)

print("\n" + "=" * 60)
```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
stud stud stud stud
organ organ organ organ
```

VOCABULARY AFTER NORMALIZATION
['connect', 'organ', 'studi']
Vocabulary Size: 3

PROCESSING TIME
Before Normalization: 0.005489 seconds
After Normalization: 0.00114 seconds

MORPHOLOGICAL NORMALIZATION
connected -> connect
connection -> connect
connecting -> connect
connectivity -> connect
studies -> studi
studied -> studi
studying -> studi
study -> studi
organize -> organ
organized -> organ
organizer -> organ
organization -> organ

COMPARISON
Vocabulary before: 12
Vocabulary after: 3
Vocabulary reduction: 9

CONCLUSION
Morphological normalization is performed before
feature extraction.
Related morphological forms can share a common stem.
This reduces redundant vocabulary and can improve
machine-learning feature representation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	3	3	18	88
2	4	4	3	3	4	18	
3	4	4	3	4	3	18	
4	4	3	3	3	4	17	
5	3	3	3	4	4	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____